Java OOP Interview Notes

Concise explanations, Java examples, e-commerce project examples, and interview questions

Topics covered: Encapsulation, Abstraction, Inheritance, Polymorphism, Runtime Polymorphism, Interface, Abstract Class, Interface vs Abstract Class, Overloading vs Overriding.

1. Encapsulation

Definition: Encapsulation means binding data and related methods into a class and restricting direct access to internal data. In Java, it is mainly achieved using private fields and controlled public methods.

Easy memory: Protect the data + control how it is modified.

Java example:

```
public class Product {
    private int stock;

    public int getStock() {
        return stock;
    }

    public void reduceStock(int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Invalid quantity");
        }
        if (quantity > stock) {
            throw new IllegalStateException("Insufficient stock");
        }
        stock -= quantity;
    }
}
```

Real-time e-commerce example: In Product Service, **stock** is private. Instead of directly changing it, the application calls **reduceStock(quantity)**, which validates the quantity and prevents invalid inventory values. For business-critical fields such as order status, methods such as **confirmOrder()** and **cancelOrder()** are better than unrestricted setters.

> **Interview answer:**
> Encapsulation is the process of binding data and related methods into a single class and restricting direct access to the internal data. We mainly achieve it using private fields and controlled methods. In our e-commerce application, Product stock is private and is modified through reduceStock(), where we can validate the quantity before updating it.

Setters and encapsulation

Using setters does not automatically violate encapsulation. DTOs commonly use getters and setters because their main purpose is data transfer. For business-critical domain fields, meaningful methods with validation are preferable.

Data hiding vs encapsulation: A private field provides data hiding. Encapsulation is the broader design of keeping state and the operations that control that state together.

2. Abstraction

Definition: Abstraction means hiding internal implementation details and exposing only the required functionality. In Java, it is mainly achieved using interfaces and abstract classes.

Easy memory: Hide HOW, expose WHAT.

Java example:

```
public interface OrderService {
    OrderResponse createOrder(OrderRequest request);
}

@Service
public class OrderServiceImpl implements OrderService {
    @Override
    public OrderResponse createOrder(OrderRequest request) {
        // validate product
        // save order
        // publish Kafka event
        return response;
    }
}
```

Real-time e-commerce example: The OrderController calls **orderService.createOrder()**. It does not need to know the internal steps such as validation, database operations, or Kafka event publishing.

> **Interview answer:**
> In our e-commerce application, we use abstraction in the service layer. OrderService exposes operations such as createOrder(), while OrderServiceImpl contains the actual business logic. The controller works with the service contract without knowing the internal implementation.

3. Inheritance

Definition: Inheritance is an OOP concept where one class acquires the properties and behaviours of another class. It supports reuse and represents an IS-A relationship. Java class inheritance uses the extends keyword.

Easy memory: IS-A relationship + common parent behaviour/state.

Java example:

```
@MappedSuperclass
public abstract class BaseEntity {
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}

@Entity
public class Order extends BaseEntity {
    private Long orderId;
}

@Entity
public class Product extends BaseEntity {
    private Long productId;
}
```

Real-time e-commerce example: Order, Product and Payment can extend a BaseEntity containing common audit fields such as createdAt and updatedAt, avoiding duplication.

> **Interview answer:**
> In our e-commerce application, a BaseEntity can contain common fields such as createdAt and

updatedAt. Order, Product and Payment can extend BaseEntity so those common properties do not need to be repeated.

4. Polymorphism

Definition: Polymorphism means one contract or method can have multiple implementations or behaviours. Java commonly demonstrates compile-time polymorphism through overloading and runtime polymorphism through overriding.

Easy memory: One contract, multiple behaviours.

Payment example:

```
interface PaymentService {
    void processPayment(double amount);
}

class UpiPayment implements PaymentService {
    @Override
    public void processPayment(double amount) {
        System.out.println("UPI Payment");
    }
}

class CardPayment implements PaymentService {
    @Override
    public void processPayment(double amount) {
        System.out.println("Card Payment");
    }
}
```

Real-time e-commerce example: PaymentService can have UPI, Card and NetBanking implementations. The caller works with the common contract while each implementation provides its own processing logic.

Interview answer:
In an e-commerce application, we can define a common PaymentService interface and provide UPI, Card and NetBanking implementations. The application can work with the PaymentService reference while the actual implementation provides the required behaviour.

5. Compile-time Polymorphism - Method Overloading

Method overloading means using the same method name with a different parameter list. The compiler determines which matching method to call.

```
void createOrder(Long productId) { }

void createOrder(Long productId, int quantity) { }
```

Remember: Overloading = compile-time polymorphism.

6. Runtime Polymorphism - Method Overriding

Runtime polymorphism occurs when a parent class or interface reference refers to a child implementation and the overridden method that executes depends on the actual object at runtime.

```
PaymentService payment = new UpiPayment();
payment.processPayment(5000);  // UPI implementation

payment = new CardPayment();
payment.processPayment(5000);  // Card implementation
```

Why is it called runtime? The compiler verifies that processPayment() is valid for PaymentService. The actual overridden implementation that executes is selected according to the runtime object.

> **Interview answer:**
> Runtime polymorphism is achieved through method overriding. A parent class or interface reference can refer to a child object, and the overridden method that gets executed is determined at runtime based on the actual object.

7. Interface

An interface defines a contract that specifies what a class should do, while implementing classes provide the actual behaviour. It is commonly used for abstraction, loose coupling and multiple implementations.

```
public interface PaymentService {
    void processPayment(double amount);

    default void validateAmount(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid amount");
        }
    }
}
```

Modern Java point: Interfaces can contain abstract methods, default methods, static methods and private helper methods. A class can implement multiple interfaces.

E-commerce example: PaymentService can be implemented by UpiPaymentService, CardPaymentService and NetBankingPaymentService.

> **Interview answer:**
> An interface defines a contract and allows multiple classes to provide different implementations. I use interfaces in service-layer design to achieve abstraction and loose coupling.

8. Abstract Class

An abstract class cannot be instantiated directly. It can contain abstract methods, concrete methods, instance state and constructors. It is useful when closely related subclasses need a common base.

```
public abstract class PaymentProcessor {
    protected String merchantId;

    public PaymentProcessor(String merchantId) {
        this.merchantId = merchantId;
    }

    public void validateAmount(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid amount");
        }
    }

    public abstract void processPayment(double amount);
}
```

E-commerce example: UPI and Card processors may share merchant information, common validation and logging while implementing their own processPayment() logic.

> **Interview answer:**
> An abstract class is useful when related classes need shared state, constructors and common

implementation while still requiring subclasses to implement specific behaviour.

9. Interface vs Abstract Class

Interface	Abstract Class
Primarily defines a contract	Primarily provides a common base
Class uses implements	Class uses extends
Can provide default methods	Can provide concrete methods
No normal per-object instance state	Can hold instance state
No constructor for implementation object initialization	Can have constructors
A class can implement multiple interfaces	A class can extend only one class
Good for loose coupling / capabilities	Good for related classes sharing state and logic

Important question: If interfaces have default methods, why use an abstract class?

Default methods allow an interface to share behaviour, but an abstract class can additionally maintain per-object instance state, use constructors and provide a common base for closely related classes. Use an interface mainly for a contract/loose coupling; use an abstract class when subclasses need shared state and common implementation.

10. Key Differences

Encapsulation vs Abstraction

Encapsulation: Protect data and control how it is modified. Example: Product.stock -> private -> reduceStock().

Abstraction: Hide implementation details and expose required functionality. Example: OrderController -> OrderService -> OrderServiceImpl.

Overloading vs Overriding

Overloading	Overriding
Same method name, different parameters	Same method signature, new implementation
Compile-time polymorphism	Runtime polymorphism
Usually in the same class	Requires inheritance/interface implementation relationship

OOP Interview Questions & Answers

1. What are the four pillars of OOP?

Encapsulation, Abstraction, Inheritance and Polymorphism.

2. What is encapsulation and where did you use it?

Encapsulation protects internal state and controls modification through methods. In our e-commerce Product domain, stock can be private and updated through reduceStock() with validation.

3. Do getters and setters break encapsulation?

No. Fields are still private and access is controlled through class methods. DTOs commonly use setters. For business-critical state, meaningful methods with validation are preferable to unrestricted setters.

4. What is the difference between data hiding and encapsulation?

Data hiding restricts direct access to data, for example with private fields. Encapsulation is broader: it groups state and related behaviour and controls how that state is accessed or modified.

5. What is abstraction and where did you use it?

Abstraction hides implementation details and exposes required operations. In our service layer, OrderService exposes createOrder(), while OrderServiceImpl handles database and Kafka details.

6. Encapsulation vs abstraction?

Encapsulation focuses on protecting state and controlling modification. Abstraction focuses on hiding implementation complexity and exposing only required functionality.

7. What is inheritance?

Inheritance allows a child class to acquire parent properties and behaviour and represents an IS-A relationship. A BaseEntity can provide common audit fields to Order and Product.

8. Why should inheritance not be used only for code reuse?

Inheritance creates a strong IS-A relationship. If that relationship is not logically correct, composition is usually a better design even if inheritance could reduce duplicate code.

9. What is polymorphism?

Polymorphism means one contract or method can have multiple behaviours. Java provides compile-time polymorphism through overloading and runtime polymorphism through overriding.

10. What is runtime polymorphism?

A parent/interface reference can refer to a child object, and the overridden method that executes is determined by the actual object at runtime.

11. What is method overloading?

Same method name with a different parameter list. The compiler resolves the call, so it is compile-time polymorphism.

12. What is method overriding?

A subclass or implementation provides its own implementation of a method with the same signature. It enables runtime polymorphism.

13. Can we override static methods?

No. Static methods belong to the class and are hidden, not overridden in the runtime-polymorphism sense.

14. Can we override private methods?

No. Private methods are not inherited by subclasses, so they cannot be overridden.

15. Can we override final methods?

No. A final method cannot be overridden.

16. What is an interface?

An interface primarily defines a contract. Implementing classes provide behaviour. It is commonly used for abstraction, loose coupling and multiple implementations.

17. Can an interface have method implementation?

Yes. Modern Java interfaces can have default methods, static methods and private helper methods in addition to abstract methods.

18. Why were default methods introduced?

A major reason was interface evolution: new behaviour could be added to an interface without forcing every existing implementation to immediately implement the new method.

19. Can an interface have variables?

Interface fields are implicitly public, static and final. They are constants, not normal per-object instance fields.

20. Can we create an object of an interface?

No. We create an object of an implementing class and can refer to it using the interface type.

21. Can a class implement multiple interfaces?

Yes. This is one way Java supports multiple inheritance of type/behaviour contracts.

22. What is an abstract class?

An abstract class cannot be instantiated directly and can contain abstract methods, concrete methods, constructors and instance state.

23. Can an abstract class have a constructor?

Yes. The constructor is used when a subclass object is created and can initialize the common parent state.

24. Can an abstract class have no abstract methods?

Yes. A class may be declared abstract even if it contains no abstract methods, for example to prevent direct instantiation or provide a common base.

25. Can an abstract class implement an interface?

Yes. It may implement all, some, or none of the interface's abstract methods. A concrete subclass must eventually implement any remaining required methods.

26. Interface vs abstract class - when do you choose each?

Use an interface primarily for a contract, loose coupling and multiple implementations. Use an abstract class when closely related subclasses need shared instance state, constructors or substantial common implementation.

27. If interfaces have default methods, why do we still need abstract classes?

Default methods share behaviour, but abstract classes can also hold per-object instance state, use constructors and form a common stateful base for related classes.

28. Where did you use polymorphism in your e-commerce project?

A PaymentService contract can have UPI, Card and NetBanking implementations. The caller works with PaymentService while the selected implementation provides payment-specific behaviour.

29. What is dynamic method dispatch?

It is the mechanism Java uses for runtime polymorphism: the overridden instance method is selected based on the actual runtime object.

30. Can constructors be overridden?

No. Constructors are not inherited, so they cannot be overridden. They can be overloaded within the same class.

30-Second OOP Revision

Encapsulation - Protect data and control how it is modified.

Abstraction - Hide implementation and expose required functionality.

Inheritance - Child acquires parent properties/behaviour; IS-A relationship.

Polymorphism - One contract or method can have multiple behaviours.

Overloading - Compile-time polymorphism.

Overriding - Runtime polymorphism.

Interface - Contract + loose coupling + multiple implementations.

Abstract Class - Common base + shared state + common implementation.

E-commerce memory map

Encapsulation: Product.stock -> reduceStock()

Abstraction: OrderController -> OrderService -> OrderServiceImpl

Inheritance: BaseEntity -> Order / Product / Payment

Polymorphism: PaymentService -> UPI / Card / NetBanking